SYSTEM DESIGN · DRAFT 1 · 27 SEPTEMBER 2026

Mother of Lemmatizers

One lemmatizer for every language in the Glottolog list: it recognises the language of each text segment, lemmatizes with curated grammar rules where they exist, and, for a language it has never seen, finds its relatives and derives a provisional grammar from them using the regularities of language change.

established knowledge: curated rules, treebanks, documented grammars

inferred: transferred, induced or reconstructed, always carries a confidence

Overview

The input is raw text in any language and script. The output is the same text with a lemma for every word, plus the recognised language, the grammar that produced each lemma and a confidence. Every text goes through the same recognition stage, then is routed to one of three tiers depending on what the system knows about the language:

Tier A · rules exist

Language recognised, curated grammar available

Rule-based analyser proposes all candidate lemmas; a contextual neural model chooses among them. Highest accuracy, fully explainable.

Tier B · known, no rules

Language recognised, no machine-readable grammar

Rules are extracted from grammar books in the document catalogue and induced from raw text, then compiled into the same rule format.

Tier C · unknown language

Recogniser rejects all known labels

Closest relatives are located in the family tree; their grammars are carried over through learned sound correspondences, fitted to the text, and checked against typological universals.

Grammars produced in tiers B and C are provisional. They are stored with their provenance and confidence, used immediately, and promoted to Tier A only after validation: automatic checks first, then a linguist's review. Each language processed this way improves the system for its neighbours in the tree.

Architecture schema

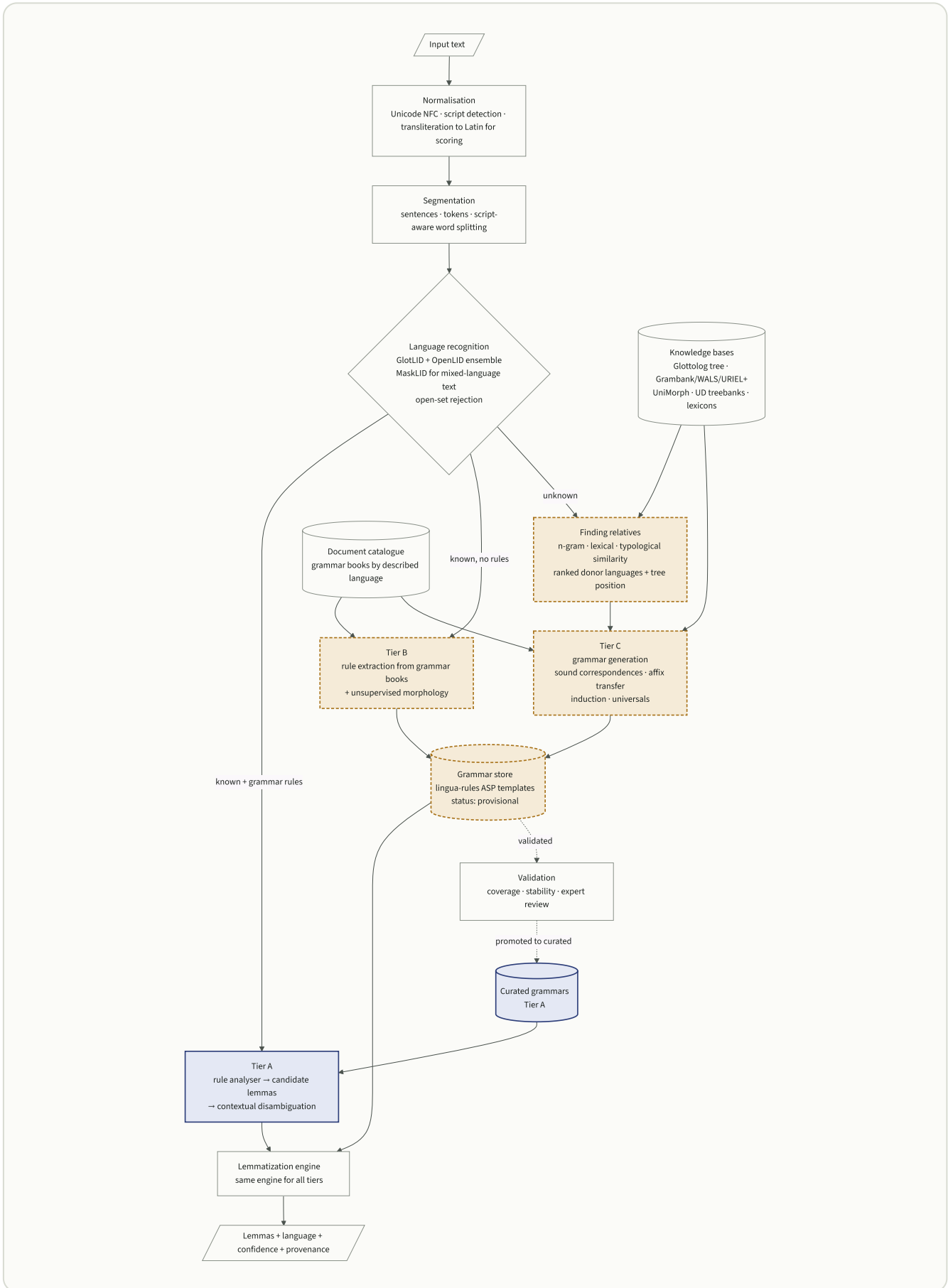


Figure 1. Components and data flow. Solid blue marks curated knowledge; dashed ochre marks inferred knowledge that carries a confidence and waits for validation.

Components

COMPONENT	JOB	METHOD
Normalisation	Make text comparable across sources	Unicode NFC, removal of zero-width characters, script detection by Unicode script property (ISO 15924), romanisation (uroman-style) used only for similarity scoring, never for output
Segmentation	Sentences and word tokens	Rule-based for spaced scripts; dictionary/statistical segmentation for scripts without spaces (Thai, Chinese, Japanese)
Language recognition	Language label per segment, or "unknown"	Ensemble of fastText classifiers, code-switching detection, calibrated open-set rejection (section 3)
Finding relatives	For an unknown language: donors and tree position	Character n-gram, lexical (cognate) and typological similarity combined by a learned ranker (section 4)
Grammar store	One format for all grammars	Paradigm and rule templates in the lingua-rules ASP format, with confidence, provenance and status per rule
Lemmatization engine	Word → lemma in context	Rule analyser generates candidates; neural edit-script model scores them in context; guesser handles unknown words
Validation	Decide when inferred rules can be trusted	Corpus coverage, bootstrap stability, round-trip generation, held-out checks, expert review queue

Language recognition

Recognition answers two questions for every segment: which known language is it, and is it perhaps none of them. The second question matters most here, because a closed-set classifier always returns some label, even for a language it has never seen.

Algorithm

1. **Script first.** The dominant Unicode script restricts the candidate set (a Cyrillic text cannot be Hindi). Script is part of the label, as in GlotLID (`kaz_Cyrl` vs `kaz_Latn`).
2. **Closed-set classification.** GlotLID v3 (fastText, 2,102 language–script labels, trained on 1,832 languages) returns the top-k labels with probabilities. OpenLID-v3 is added as a second vote because it was tuned for precision among closely related languages. Probabilities are calibrated per label (temperature scaling on held-out data) so that thresholds mean the same thing for every language.
3. **Mixed-language text.** Segments are scored in sliding windows. MaskLID removes the features of the dominant language and re-classifies the remainder, which reveals an embedded second language without any extra training.

4. **Open-set rejection.** A segment is marked *unknown* when any of these holds: the calibrated top probability is below τ_p; the entropy of the top-k distribution is above τ_H; or the text is an outlier for the winning label, i.e. its character n-gram perplexity under that language's model is far above the perplexity of that language's own validation texts (z-score above τ_z). The third test catches the dangerous case: a confident but wrong label for a relative of the true language.
5. **Document-level smoothing.** Neighbouring segments rarely switch language, so labels are smoothed with a simple HMM over segments (switch penalty); isolated low-confidence flips are overruled.

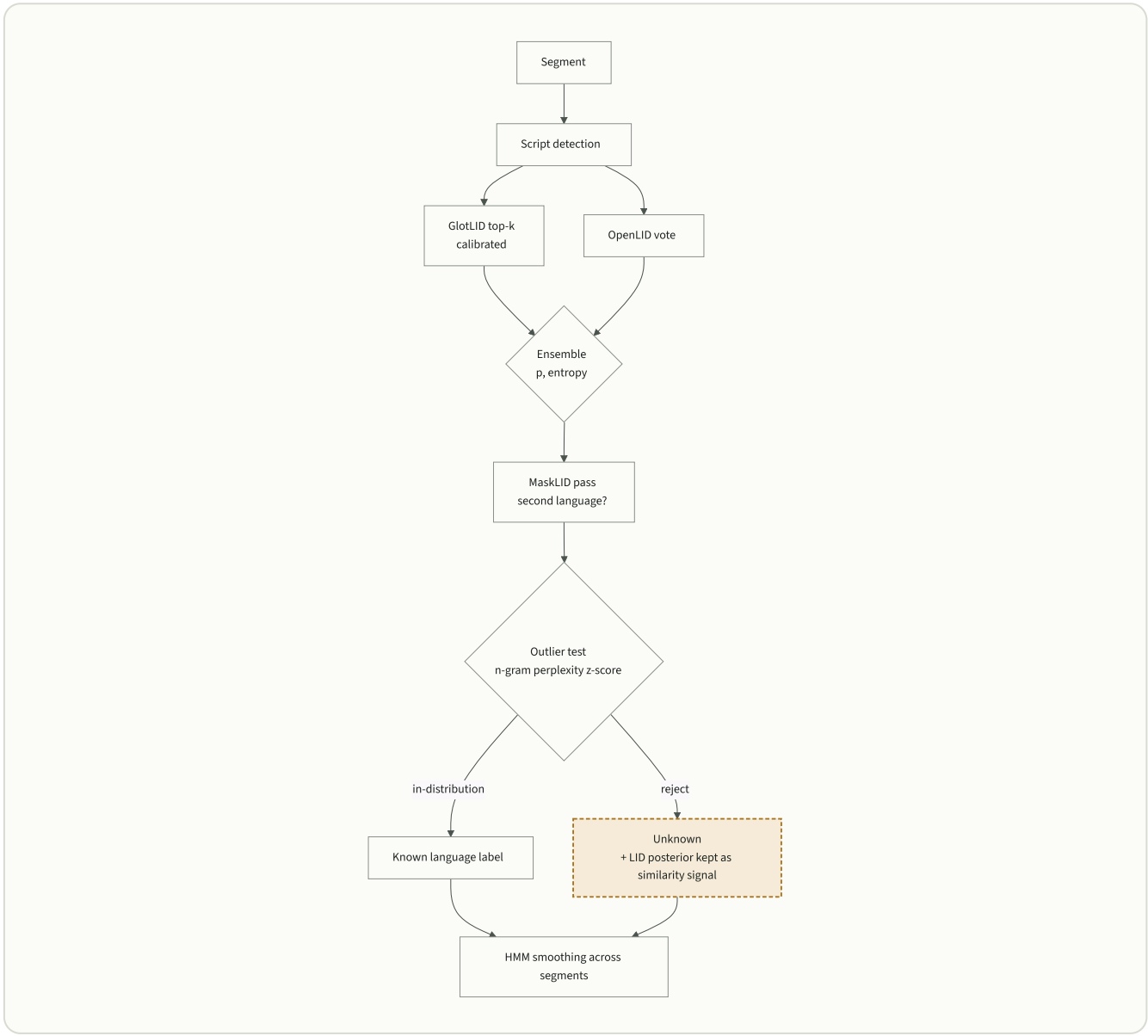


Figure 2. Recognition pipeline per segment. A rejected segment keeps its full probability distribution: it becomes the first similarity signal for finding relatives.

```
# language recognition for one document
def recognise(doc):
    segments = segment(normalise(doc))
    labels = []
    for seg in segments:
        script = dominant_script(seg)
        dist = ensemble(glotlid.topk(seg, 20), openlid.topk(seg, 20), allowed_script=script)
        dist = calibrate(dist)                                  # per-label temperature
        top, p = dist.best()
        z = perplexity_zscore(seg, ngram_model[top])           # outlier for the winning
label?
        if p < TAU_P or entropy(dist) > TAU_H or z > TAU_Z:
            labels.append(Unknown(posterior=dist, script=script))
        else:
            second = masklid(seg, dominant=top)                 # code-switching
            labels.append(Known(top, p, embedded=second))
    return hmm_smooth(labels, switch_penalty=SWITCH)
```

Finding relatives of an unknown language

For an unknown language the system needs a small set of *donor* languages whose grammars can be carried over, and an estimate of where the language sits in the Glottolog family tree. No single signal is reliable: shared vocabulary may be borrowed, and similar spelling may only reflect a shared orthographic tradition. The ranker therefore combines four independent signals, following the LangRank approach of learning which features predict successful transfer.

SIGNAL	HOW IT IS COMPUTED	WHAT IT CAPTURES
LID posterior	The rejected distribution from recognition, aggregated over all segments	Surface resemblance, cheap and immediately available
Character n-gram profile	Rank-order distance (Cavnar–Trenkle "out-of-place") and cosine distance of 1–5-gram profiles against reference corpora of candidate languages	Phonotactics and orthography
Lexical cognacy	The target's most frequent word types are aligned with candidate lexicons (SCA alignment); LexStat-style scoring rewards *recurrent* sound correspondences over chance similarity; share of likely cognates in core vocabulary	Genealogical relatedness; the strongest signal when enough text exists
Typological fingerprint	Features estimated from the raw text (mean morphs per word from unsupervised segmentation, prefix vs suffix share, reduplication rate, word-order cues from frequent function words) compared with Grambank/URIEL+ vectors	Structural similarity; useful when the relatives are distant or unknown

A gradient-boosted ranker (as in LangRank) combines the signals with priors (script, geography of candidate languages, data size of each candidate's grammar) and returns ranked donors. Tree position is estimated as the lowest common ancestor in Glottolog of the top donors, weighted by their scores. When no donor passes

the cognacy threshold, the language is treated as a potential isolate: Tier C then relies only on unsupervised induction and typological priors.

```
# rank donor languages for an unknown language U
def find_relatives(U_text, posterior, candidates):
    freq_types = most_frequent_types(U_text, n=2000)
    for L in candidates:                                  # pre-filtered by script, geography,
posterior
        f_post  = posterior.get(L, 0)
        f_ngram = out_of_place(ngram_profile(U_text), profile[L])
        f_cog   = lexstat_cognate_share(freq_types, lexicon[L])   # recurrent correspondences
only
        f_typo  = cosine(estimate_typology(U_text), grambank_vec[L])
        score[L] = ranker.predict([f_post, f_ngram, f_cog, f_typo, prior(L)])
    donors = top_k(score, k=5)
    position = weighted_lca(glottolog, donors, score)
    return donors, position, is_isolate(f_cog_max < TAU_COG)
```

Lemmatizing with existing rules (Tier A)

Where curated grammar exists, the system combines exact rules with a statistical model that reads context. Rules guarantee that every lemma is a legitimate form of the language; the model decides among ambiguous analyses (German *Weg* as noun "way" vs particle *weg*, Russian *стали* as "steel" or "became").

1. **Analysis.** Each token is run through the rule analyser (paradigm templates and alternation rules compiled from the grammar store). The result is a set of candidate analyses: lemma + part of speech + features.
2. **Lexicon check.** Candidates whose lemma is in the lexicon are preferred; unknown lemmas are allowed but penalised.
3. **Unknown words.** A guesser strips the most productive affixes (productivity = number of distinct stems observed with the affix) and proposes candidate lemmas ranked by paradigm fit.
4. **Contextual choice.** A neural edit-script lemmatizer (the approach of the Universal Lemmatizer and Stanza/UDPipe, trained on Universal Dependencies) scores every candidate in its sentence. The output is constrained to the rule candidates whenever at least one exists, so the model cannot invent non-words.

Known language without rules (Tier B)

For most of the languages in our catalogue a grammar exists only as a book. Tier B converts those books and the available raw text into rules.

1. **Retrieve.** Grammar books, sketches and textbooks whose described language is the target are fetched from the document catalogue.
2. **Extract.** The local language model reads the paradigm tables and the morphology chapters and emits structured rules: affix, slot, features, conditions, examples. Interlinear examples (IGT) are parsed into morpheme–gloss pairs and used as test cases. Research on learning from grammar books (MTOB) shows that models profit mostly from examples and tables rather than prose explanations, so extraction concentrates on those.

3. **Induce.** Independently, unsupervised segmentation (Morfessor FlatCat) and paradigm clustering on raw text propose affixes and paradigms.
4. **Reconcile.** Extracted rules act as a prior; the induced segmentation provides evidence. Rules that the text confirms get high confidence; rules the text never exercises remain but are flagged; frequent unexplained patterns become new rule candidates.
5. **Compile.** The result is written into the grammar store in the lingua-rules template format, so the Tier A engine can use it unchanged.

Grammar generation for an unknown language (Tier C)

This is where the system uses the laws of language change. The working assumption is the comparative method's: related languages descend from a common ancestor, and the differences between them are largely *regular*. If the regular correspondences between the unknown language and its relatives can be learned, the relatives' morphology can be mapped onto the unknown language and then tested against its text.

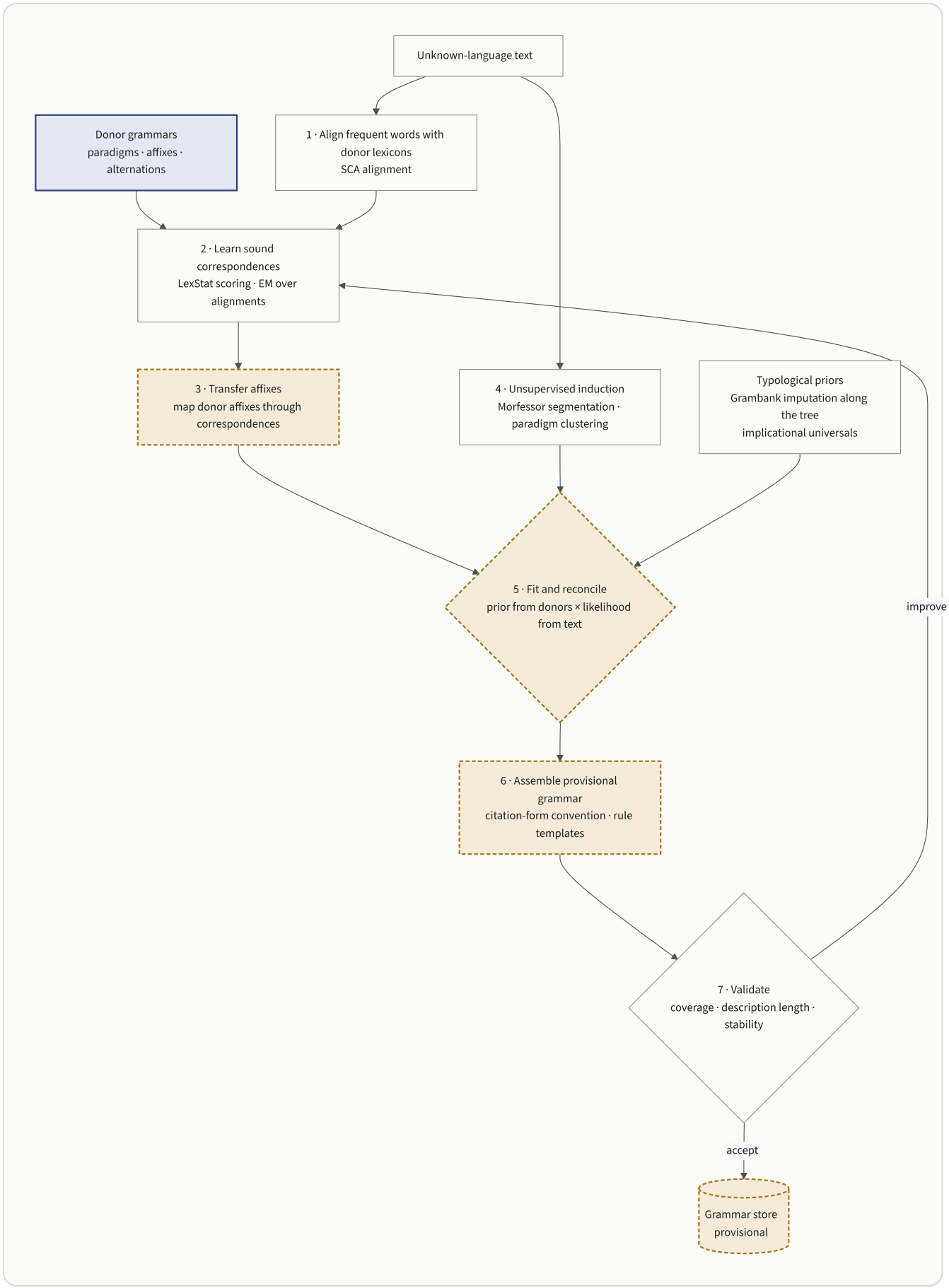


Figure 3. The grammar generation loop. Steps 2 to 7 repeat until the grammar's coverage and stability stop improving.

1 · Lexical alignment

The 2,000 most frequent word types of the unknown text are aligned with the lexicons of the donors using sound-class alignment (SCA), which compares words by classes of similar sounds rather than exact letters. Where the orthography is not phonemic, a grapheme-to-phoneme step (Epitran-style) converts both sides to IPA first. Only pairs above a similarity threshold enter the next step.

2 · Sound correspondences

From the aligned pairs the system learns a correspondence table: which sound of the donor appears as which sound in the unknown language, in which context (e.g. donor p → target f at the start of a word; donor k → tʃ before front vowels). The LexStat principle is applied: a correspondence counts only if it recurs across many word pairs far more often than in randomly shuffled pairings, which filters out chance lookalikes and most loanwords. Estimation is an EM loop: correspondences re-score the alignments, better alignments refine the correspondences. Where several donors exist, the correspondence patterns across all of them are inferred jointly, which is more robust than pairwise tables.

3 · Affix transfer

Each affix of each donor paradigm is passed through the correspondence table, producing predicted target affixes with probabilities. A prediction survives if the predicted string actually occurs at the right edge (suffix) or left edge (prefix) of many different word types in the target text. For example, if a donor marks plural with `-at` and the table maps a → e, the system looks for a productive `-et` in the target.

4 · Unsupervised induction

In parallel, Morfessor FlatCat segments the target words into stems and affixes, and paradigm clustering groups words that share a stem and differ by affix sets. This step sees only the target text, so it also finds morphology that the donors do not have.

5 · Fitting and reconciliation

Transferred affixes are the prior, induced segmentation is the evidence. A hypothesis is scored by how much it shortens the description of the corpus (minimum description length: the grammar plus the corpus encoded with it). Typological priors decide close calls: missing Grambank features of the target are imputed from its position in the tree (phylogenetically weighted nearest neighbours), and implicational universals exclude improbable combinations. A suffixing-preference prior applies where the evidence is balanced, because suffixing is cross-linguistically far more common than prefixing.

6 · Assembly

Accepted affixes and alternations are organised into paradigms and written as lingua-rules templates. The lemma convention (which form counts as the dictionary form: nominative singular, infinitive, third person singular…) is copied from the closest donor, because that is what grammars of the family use. Every rule records its origin (transferred from which donor, induced, or both), its confidence and the number of supporting word types.

7 · Validation and iteration

The grammar is judged on the target text itself: share of tokens that receive an analysis (coverage), description length, stability of the rules under bootstrap resampling of the text, and round-trip consistency (generate forms from lemmas and check that they occur). Weak rules are dropped or re-estimated and the loop repeats. The final grammar is stored as *provisional*; it becomes curated only after review.

```
# Tier C: derive a provisional grammar for unknown language U
def generate_grammar(U_text, donors, position):
    words = frequent_types(U_text, 2000)
    induced = morfessor_flatcat(U_text) + paradigm_clusters(U_text)
    priors  = impute_grambank(position) + universals()
    corr = init_correspondences()                                  # identity + common sound
changes
    for it in range(MAX_ITER):
        pairs = [align_sca(w, lexicon[d]) for d in donors for w in words]
        pairs = [p for p in pairs if lexstat_score(p, corr) > TAU_ALIGN]
        corr  = estimate_correspondences(pairs, shuffled_baseline=True)    # EM step
        transferred = [map_affix(a, corr) for d in donors for a in affixes(d)]
        transferred = [a for a in transferred if edge_support(a, U_text) ⩾ MIN_TYPES]
        grammar = fit_mdl(prior=transferred, evidence=induced, typology=priors)
        score = validate(grammar, U_text)                          # coverage, MDL, bootstrap
stability
        if score.gain < EPS:
            break
    grammar.lemma_convention = lemma_convention(closest(donors))
    return grammar.with_status("provisional", provenance=donors, confidence=score)
```

Laws of language evolution the system relies on

"Common linguistic laws" is made concrete as five kinds of knowledge, each encoded as data rather than as hand-written code:

PRINCIPLE	ENCODED AS	USED IN
Regularity of sound change: changes apply to all words with the same sound in the same environment	Correspondence tables learned by EM; only recurrent correspondences are accepted	Steps 1–3
Directional tendencies of sound change (lenition, palatalisation before front vowels, final devoicing, vowel reduction, assimilation)	Prior probabilities over change types, from databases of attested sound changes	Initialising and regularising the correspondence table
Genealogical inheritance: related languages share morphology inherited from their ancestor; stable features change slowly	Glottolog tree; feature imputation weighted by tree distance	Donor ranking, typological priors
Typological universals: implicational tendencies (e.g. verb-final languages tend to have postpositions; suffixing preference)	Constraints and priors over Grambank/WALS features	Step 5
Grammaticalisation paths: grammatical markers develop from lexical words along recurring paths (e.g. "go" → future)	Source → target mappings used to interpret affixes whose meaning is unclear	Labelling induced affixes with grammatical features

> Neural reconstruction models (cognate reflex prediction, protoform reconstruction) can replace the EM correspondence learner once enough cognate sets exist for a family; they were the strongest systems in the SIGTYP 2022 shared task. The EM learner stays as the fallback for families with little data.

Input and output

The same JSON contract serves every tier, so callers never need to know how a lemma was produced:

```
POST /lemmatize
{ "text": "Kharia text or anything else…", "hint_language": null }

200 OK
{
  "segments": [
    { "span": [0, 118],
      "language": { "glottocode": "khar1287", "iso639_3": "khr", "status": "known",
                    "confidence": 0.93, "embedded": null },
      "tier": "B",
      "grammar": { "id": "khar1287@2026-09-27", "status": "provisional",
                   "sources": ["catalog:2932", "induced"] },
      "tokens": [
        { "form": "…", "lemma": "…", "pos": "NOUN", "features": "Case=Obl|Number=Sing",
          "confidence": 0.81, "rule": "noun_obl_sg", "origin": "extracted" }
      ] }
  ]
}
```

For an unknown language, `language.status` is `"unknown"` and the block adds `donors` (ranked relatives with scores) and `tree_position` (the estimated Glottolog node).

Mapping to our stack

Most of the knowledge the design needs is already being collected on corpus-host:

DESIGN ELEMENT	EXISTING ASSET	STATE (27 SEP 2026)
Language inventory and family tree	Glottolog language list on the host	8,624 languages
Grammar books for Tier B	Document catalogue, field "described language"	1,718 languages with a confirmed grammar
Grammar store format	lingua-rules (ASP rule templates, generators, golden tests)	Curated rules for English, German, Russian, Finnish
Rule extraction and relative ranking model	Local LLM on the RTX 5090 (qwen3.8, 128k context)	Running; already classifies catalogue documents
Retrieval of grammar documents	Corpus catalogue REST API (`/corpus/api/documents?described=…`)	Live, HTTPS
Raw text for induction and n-gram profiles	Language corpus and grammar collections	≈ 290,000 documents registered

Missing pieces: GlotLID/OpenLID models, the UD and UniMorph data, Grambank/URIEL+ vectors and an implementation of the correspondence learner (LingPy provides SCA alignment and LexStat).

Evaluation

- **Recognition:** accuracy and macro-F1 on held-out GlotLID/FLORES-style test sets; open-set rejection measured by hiding whole languages from the classifier and checking that they are rejected, and that the family of the hidden language comes first among the suggested relatives.
- **Tier A:** lemma accuracy on Universal Dependencies test treebanks, compared with Stanza/UDPipe.
- **Tier C by simulation:** the decisive test. Take a language that *has* a curated grammar and a treebank, pretend it is unknown, generate its grammar only from its relatives, and measure lemma accuracy against the treebank. Repeating this leave-one-language-out across families shows how accurate provisional grammars are as a function of relative distance and text size.
- **Tier B:** extracted rules checked against the grammar book's own interlinear examples, then against a treebank where one exists.

Limits and risks

- **Close relatives confuse recognition.** Mutually intelligible varieties (e.g. Bosnian/Croatian/Serbian, Malay/Indonesian) are routinely confused by LID; the outlier test and the second vote reduce but do not remove this.
- **Correspondences need data.** Reliable correspondence learning needs on the order of a hundred or more cognate pairs; very short texts only support n-gram and typological signals.
- **Loanwords and contact.** Borrowed vocabulary can make a language look related to its neighbour; LexStat's regularity test and the typological signal counter this but not completely.
- **Isolates and deep families.** Without close relatives, Tier C degrades to unsupervised induction plus priors. The SIGMORPHON 2020 shared task found unsupervised paradigm completion still largely unsolved, so expectations for isolates must be modest.
- **Orthography is not phonology.** Correspondences are learned over sounds; deep or inconsistent orthographies need a grapheme-to-phoneme step, which may itself be unavailable.
- **Provisional grammars can be wrong with confidence.** Every inferred lemma carries its origin and confidence, and downstream users should filter by status.

References

1. Kargaran et al. (2023). GlotLID: Language Identification for Low-Resource Languages. EMNLP. Code: github.com/cisnlp/GlotLID
2. OpenLID-v3 (2026). Improving the Precision of Closely Related Language Identification — an experience report.
3. Kargaran et al. (2024). MaskLID: Code-Switching Language Identification through Iterative Masking. ACL.
4. Lin et al. (2019). Choosing Transfer Languages for Cross-Lingual Learning (LangRank). ACL.
5. List (2012). LexStat: Automatic detection of cognates in multilingual wordlists; List (2019). Automatic Inference of Sound Correspondence Patterns across Multiple Languages. Computational Linguistics.
6. Bouchard-Côté, Hall, Griffiths, Klein (2013). Automated reconstruction of ancient languages using probabilistic models of sound change. PNAS 110(11).
7. List et al. (2022). The SIGTYP 2022 Shared Task on the Prediction of Cognate Reflexes; Kim et al. (2023). Cognate Transformer for Automated Phonological Reconstruction and Cognate Reflex Prediction.
8. Kann et al. (2020). The SIGMORPHON 2020 Shared Task on Unsupervised Morphological Paradigm Completion.
9. Grönroos et al. (2014). Morfessor FlatCat: an HMM-based method for unsupervised and semi-supervised learning of morphology. COLING. See also Morphological Segmentation for Low Resource Languages (LREC 2020).
10. Kanerva et al. Universal Lemmatizer: a sequence-to-sequence model for lemmatizing Universal Dependencies treebanks. Natural Language Engineering.

11. Chaudhary et al. (2022). AutoLEX: An Automatic Framework for Linguistic Exploration; Chaudhary et al. (2020). Automatic Extraction of Rules Governing Morphological Agreement.
12. Tanzer et al. (2024). A Benchmark for Learning to Translate a New Language from One Grammar Book (MTOB). ICLR; Aycock et al. (2024). Can LLMs really learn to translate a low-resource language from one grammar book?
13. Skirgård et al. (2023). Grambank reveals the importance of genealogical constraints on linguistic diversity. Science Advances; Bjerva et al. (2020). SIGTYP 2020 Shared Task: Prediction of Typological Features; The Role of Typological Feature Prediction in NLP and Linguistics (Computational Linguistics 2024).
14. Hammarström et al. Glottolog (glottolog.org) — language inventory and genealogical classification.

Draft design for discussion. Numbers in "Mapping to our stack" come from the corpus-host catalogue on 27 September 2026.